

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>SkillSwap Campus — Onboarding</title>

<link rel="preconnect" href="https://fonts.googleapis.com">

<link
href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@400;500;600&display=swap"
rel="stylesheet">

<style>

:root{

  --bg-void: #0a0d1a;

  --bg-deep: #0e1224;

  --panel: rgba(255,255,255,0.045);

  --panel-hover: rgba(255,255,255,0.075);

  --border: rgba(255,255,255,0.09);

  --border-bright: rgba(255,255,255,0.18);

  --blue: #3b82f6;

  --cyan: #22d3ee;

  --violet: #8b5cf6;

  --pink: #ec4899;

  --text: #eef1fb;

  --muted: #8b93b0;

  --muted-dim: #5d6483;

  --good: #34d399;

  --warn: #fbbf24;

  --bad: #f87171;

  --radius: 20px;

  --radius-sm: 12px;

  --grad-primary: linear-gradient(135deg, var(--blue), var(--cyan));
```

```

--grad-teach: linear-gradient(135deg, #3b82f6, #22d3ee);
--grad-learn: linear-gradient(135deg, #8b5cf6, #ec4899);
--font-display: 'Space Grotesk', sans-serif;
--font-body: 'Inter', sans-serif;
--font-mono: 'JetBrains Mono', monospace;
}
*{box-sizing:border-box; margin:0; padding:0;}
html,body{height:100%;}
body{
  font-family: var(--font-body);
  background:
    radial-gradient(ellipse 900px 600px at 15% -10%, rgba(59,130,246,0.16), transparent 60%),
    radial-gradient(ellipse 900px 700px at 110% 10%, rgba(139,92,246,0.14), transparent 55%),
    radial-gradient(ellipse 700px 500px at 50% 110%, rgba(34,211,238,0.10), transparent 60%),
    var(--bg-void);
  color: var(--text);
  min-height:100vh;
  overflow-x:hidden;
}
.noise{position:fixed; inset:0; pointer-events:none; opacity:0.025; z-index:0;
  background-image:url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='100' height='100'%3E%3Cfilter id='n'%3E%3CfeTurbulence type='fractalNoise'
baseFrequency='0.9' numOctaves='2' stitchTiles='stitch'/%3E%3C/filter%3E%3Crect width='100%25'
height='100%25' filter='url(%23n)'/%3E%3C/svg%3E");
}
::selection{ background: rgba(59,130,246,0.35); }
::-webkit-scrollbar{width:8px; height:8px;}
::-webkit-scrollbar-thumb{background: rgba(255,255,255,0.12); border-radius:8px;}
a{color:inherit;}

/* ----- App shell ----- */
#app{ position:relative; z-index:1; min-height:100vh; display:flex; flex-direction:column; }

```

```

.topbar{
  display:flex; align-items:center; justify-content:space-between;
  padding: 20px clamp(20px,5vw,56px);
}

.brand{ display:flex; align-items:center; gap:10px; font-family:var(--font-display); font-weight:700;
font-size:18px; letter-spacing:-0.01em;}

.brand-mark{ width:30px; height:30px; position:relative; flex-shrink:0;}

.brand-mark svg{ width:100%; height:100%; display:block; }

.brand-tagline{ font-family:var(--font-mono); font-size:10.5px; letter-spacing:0.14em; color:var(--muted-dim); text-transform:uppercase; display:none;}

@media(min-width:720px){ .brand-tagline{ display:block; } }

.progress-shell{ padding: 0 clamp(20px,5vw,56px); }

.progress-track{ height:3px; width:100%; background: rgba(255,255,255,0.06); border-radius:99px;
overflow:hidden; }

.progress-fill{ height:100%; background: var(--grad-primary); border-radius:99px; transition: width
0.5s cubic-bezier(.4,0,.2,1); box-shadow: 0 0 12px rgba(59,130,246,0.6); }

.step-meta{ display:flex; justify-content:space-between; align-items:center; margin-top:8px; font-
family:var(--font-mono); font-size:11px; color:var(--muted-dim); letter-spacing:0.04em; }

main{ flex:1; display:flex; align-items:flex-start; justify-content:center; padding: 24px
clamp(16px,5vw,56px) 80px; }

.stage{ width:100%; max-width:600px; }

.stage.wide{ max-width: 840px; }

.card{
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: clamp(24px,4vw,40px);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);

```

```

    box-shadow: 0 30px 60px -20px rgba(0,0,0,0.5);

    animation: rise 0.5s cubic-bezier(.16,1,.3,1);
}

@keyframes rise{ from{ opacity:0; transform: translateY(18px) scale(0.99);} to{opacity:1;
transform:none;}}

@media (prefers-reduced-motion: reduce){ *{animation-duration:0.001s !important; transition-
duration:0.001s !important;}}

.eyebrow{ font-family:var(--font-mono); font-size:11.5px; letter-spacing:0.12em; text-
transform:uppercase; color:var(--cyan); margin-bottom:10px; display:flex; align-items:center;
gap:8px;}

.eyebrow::before{ content:""; width:16px; height:1px; background:var(--cyan); }

h1.step-title{ font-family:var(--font-display); font-weight:600; font-size: clamp(24px,3.4vw,32px);
line-height:1.15; letter-spacing:-0.01em; margin-bottom:8px;}

.step-sub{ color: var(--muted); font-size:14.5px; line-height:1.55; margin-bottom:28px; max-
width:52ch;}

label.field-label{ display:block; font-size:13px; font-weight:600; color:var(--text); margin-bottom:7px;
}

label.field-label .opt{ font-weight:400; color:var(--muted-dim); font-size:11.5px; margin-left:6px;}

.field-group{ margin-bottom:20px; }

input[type=text], input[type=email], input[type=password], input[type=url], input[type=time],
textarea, select{

    width:100%; background: rgba(255,255,255,0.04); border:1px solid var(--border); color:var(--text);

    padding: 13px 15px; border-radius: var(--radius-sm); font-size:14.5px; font-family: var(--font-body);

    transition: border-color .2s, background .2s, box-shadow .2s; outline:none;
}

input::placeholder, textarea::placeholder{ color: var(--muted-dim); }

input:focus, textarea:focus, select:focus{ border-color: var(--blue); background:
rgba(59,130,246,0.06); box-shadow: 0 0 0 3px rgba(59,130,246,0.15); }

input.err, textarea.err{ border-color: var(--bad) !important; box-shadow: 0 0 0 3px
rgba(248,113,113,0.12) !important; }

select{ appearance:none; background-image: url("data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' width='12' height='8'%3E%3Cpath d='M1 1l5 5 5-5'
stroke='%238b93b0' stroke-width='1.6' fill='none' fill-rule='evenodd'/%3E%3C/svg%3E");
background-repeat:no-repeat; background-position: right 15px center; padding-right:36px;}

```

```

textarea{ resize:vertical; min-height:96px; line-height:1.5; }

.hint{ font-size:12px; color:var(--muted-dim); margin-top:6px; }

.errmsg{ font-size:12.5px; color: var(--bad); margin-top:6px; display:flex; align-items:center; gap:5px;
}

.charcount{ font-size:11.5px; color:var(--muted-dim); text-align:right; margin-top:5px; font-
family:var(--font-mono);}

.charcount.over{ color:var(--bad); }


.pw-wrap{ position:relative; }

.pw-toggle{ position:absolute; right:13px; top:50%; transform:translateY(-50%); background:none;
border:none; color:var(--muted); cursor:pointer; font-size:12px; font-family:var(--font-mono);
padding:4px; }

.pw-meter{ display:flex; gap:5px; margin-top:9px; }

.pw-meter span{ height:4px; flex:1; border-radius:99px; background: rgba(255,255,255,0.08);
transition: background .3s; }

.pw-strength-label{ font-size:11.5px; margin-top:6px; font-family:var(--font-mono); }

.pw-reqs{ display:grid; grid-template-columns:1fr 1fr; gap:6px 14px; margin-top:12px; }

.pw-req{ font-size:12px; color:var(--muted-dim); display:flex; align-items:center; gap:6px;
transition:color .2s; }

.pw-req.met{ color:var(--good); }

.pw-req .dot{ width:14px; height:14px; border-radius:50%; border:1.5px solid currentColor;
display:flex; align-items:center; justify-content:center; font-size:9px; flex-shrink:0; }


.checkline{ display:flex; align-items:flex-start; gap:10px; font-size:13px; color:var(--muted); margin-
bottom:12px; cursor:pointer; }

.checkline input{ width:16px; height:16px; margin-top:2px; accent-color: var(--blue); flex-shrink:0; }


/* buttons */

.btn{ font-family: var(--font-body); font-weight:600; font-size:14.5px; border-radius: var(--radius-
sm); padding: 13px 22px; border:none; cursor:pointer; display:inline-flex; align-items:center; justify-
content:center; gap:8px; transition: transform .15s, box-shadow .15s, opacity .15s, background .2s; }

.btn:active{ transform: scale(0.97); }

.btn-primary{ background: var(--grad-primary); color:#04101f; box-shadow: 0 8px 24px -8px
rgba(59,130,246,0.55); }

.btn-primary:hover{ box-shadow: 0 10px 28px -6px rgba(59,130,246,0.7); }

```

```

.btn-primary:disabled{ opacity:0.4; cursor:not-allowed; box-shadow:none; }

.btn-ghost{ background: rgba(255,255,255,0.05); color: var(--text); border:1px solid var(--border); }

.btn-ghost:hover{ background: rgba(255,255,255,0.09); }

.btn-text{ background:none; color: var(--muted); padding: 10px 6px; }

.btn-text:hover{ color: var(--text); }

.btn-block{ width:100%; }

.actions-row{ display:flex; align-items:center; justify-content:space-between; margin-top:30px;
gap:12px; }

.actions-row .left-actions{ display:flex; align-items:center; gap:6px; }

/* OTP */

.otp-row{ display:flex; gap:10px; justify-content:center; margin: 26px 0 10px; }

.otp-row input{ width:46px; height:56px; text-align:center; font-size:22px; font-family:var(--font-
mono); font-weight:600; padding:0; }

.resend-line{ text-align:center; font-size:12.5px; color:var(--muted-dim); margin-top:14px; }

.resend-line button{ background:none; border:none; color: var(--cyan); font-weight:600;
cursor:pointer; font-size:12.5px; }

.resend-line button:disabled{ color: var(--muted-dim); cursor:not-allowed; }

.demo-code-banner{ background: rgba(34,211,238,0.08); border:1px dashed rgba(34,211,238,0.35);
border-radius: var(--radius-sm); padding:12px 14px; font-size:12.5px; color:var(--cyan); margin-
bottom:22px; font-family:var(--font-mono); text-align:center; }

/* photo */

.photo-row{ display:flex; align-items:center; gap:18px; margin-bottom:8px; }

.avatar-circle{ width:84px; height:84px; border-radius:50%; background: rgba(255,255,255,0.05);
border:2px dashed var(--border-bright); display:flex; align-items:center; justify-content:center;
color:var(--muted-dim); font-size:11px; flex-shrink:0; overflow:hidden; text-align:center; }

.avatar-circle img{ width:100%; height:100%; object-fit:cover; }

.avatar-circle.filled{ border-style:solid; }

/* username availability */

.avail-tag{ font-size:12px; font-family:var(--font-mono); display:flex; align-items:center; gap:5px;
margin-top:7px; }

.avail-tag.ok{ color: var(--good); }

```

```

.avail-tag.taken{ color: var(--bad); }

.avail-tag.checking{ color: var(--muted-dim); }


/* two col grid */

.grid2{ display:grid; grid-template-columns:1fr 1fr; gap:16px; }

@media(max-width:520px){ .grid2{ grid-template-columns:1fr; } }


/* chips / pills */

.chip-field{ display:flex; flex-wrap:wrap; gap:8px; }

.chip{ padding:8px 14px; border-radius:99px; border:1px solid var(--border); background:
rgba(255,255,255,0.03); font-size:13px; color:var(--muted); cursor:pointer; transition: all .15s; user-
select:none; }

.chip:hover{ border-color: var(--border-bright); color:var(--text); }

.chip.active{ background: rgba(59,130,246,0.16); border-color: rgba(59,130,246,0.55); color:#fff; }


/* skill search + categories */

.search-box{ position:relative; margin-bottom:18px; }

.search-box input{ padding-left:38px; }

.search-box::before{ content:'🔍'; position:absolute; left:14px; top:50%; transform:translateY(-50%);
font-size:13px; opacity:0.5; }

.skill-cat{ margin-bottom:18px; }

.skill-cat-label{ font-family:var(--font-mono); font-size:11px; letter-spacing:0.08em; text-
transform:uppercase; color:var(--muted-dim); margin-bottom:9px; }

.skill-grid{ display:flex; flex-wrap:wrap; gap:8px; }

.skill-chip{ padding:8px 14px; border-radius:10px; border:1px solid var(--border); background:
rgba(255,255,255,0.03); font-size:13px; cursor:pointer; transition: all .15s; }

.skill-chip:hover{ border-color: var(--border-bright); }

.skill-chip.teach.active{ background: linear-gradient(135deg, rgba(59,130,246,0.22),
rgba(34,211,238,0.18)); border-color: rgba(59,130,246,0.55); }

.skill-chip.learn.active{ background: linear-gradient(135deg, rgba(139,92,246,0.22),
rgba(236,72,153,0.16)); border-color: rgba(139,92,246,0.55); }

.add-custom{ display:flex; gap:8px; margin-top:14px; }

.add-custom input{ flex:1; }

```

```

.selected-panel{ margin-top:24px; border-top:1px solid var(--border); padding-top:20px; }

.selected-panel-title{ font-size:13px; font-weight:600; margin-bottom:14px; display:flex; align-
items:center; justify-content:space-between; }

.selected-panel-title .count{ font-family:var(--font-mono); font-weight:400; color:var(--muted-dim);
font-size:12px; }

.selected-item{ display:flex; align-items:center; justify-content:space-between; gap:12px;
padding:11px 14px; background: rgba(255,255,255,0.03); border:1px solid var(--border); border-
radius:var(--radius-sm); margin-bottom:8px; }

.selected-item .name{ font-size:13.5px; font-weight:500; }

.level-select{ display:flex; gap:5px; }

.level-btn{ font-size:11px; padding:5px 9px; border-radius:7px; border:1px solid var(--border);
background:transparent; color:var(--muted-dim); cursor:pointer; font-family:var(--font-mono); }

.level-btn.active{ background: var(--blue); border-color:var(--blue); color:#fff; }

.remove-x{ background:none; border:none; color:var(--muted-dim); cursor:pointer; font-size:15px;
padding:2px 4px; }

.remove-x:hover{ color:var(--bad); }

.reason-input{ margin-top:8px; }

.empty-note{ font-size:13px; color: var(--muted-dim); padding:16px; text-align:center; border:1px
dashed var(--border); border-radius: var(--radius-sm); }

/* swap diagram */

.swap-diagram{ display:flex; align-items:center; justify-content:center; gap:18px; margin: 22px 0
26px; padding:22px; background: rgba(255,255,255,0.03); border:1px solid var(--border); border-
radius:var(--radius); }

.swap-node{ text-align:center; font-size:12.5px; color:var(--muted); }

.swap-node .badge{ width:46px; height:46px; border-radius:50%; margin:0 auto 8px; display:flex;
align-items:center; justify-content:center; font-size:18px; }

.swap-node.a .badge{ background: var(--grad-teach); }

.swap-node.b .badge{ background: var(--grad-learn); }

.swap-arrows{ font-family:var(--font-mono); color:var(--cyan); font-size:20px; }

/* radio cards */

.radio-cards{ display:grid; grid-template-columns:1fr 1fr; gap:12px; }

```

```

@media(max-width:480px){ .radio-cards{grid-template-columns:1fr;} }

.radio-card{ border:1px solid var(--border); background: rgba(255,255,255,0.03); border-radius: var(--radius-sm); padding:16px; cursor:pointer; transition: all .15s; position:relative; }

.radio-card:hover{ border-color: var(--border-bright); }

.radio-card.active{ border-color: var(--blue); background: rgba(59,130,246,0.08); box-shadow: 0 0 0 1px var(--blue); }

.radio-card .rc-title{ font-weight:600; font-size:14px; margin-bottom:4px; }

.radio-card .rc-sub{ font-size:12px; color:var(--muted-dim); }

.radio-card .rc-badge{ position:absolute; top:10px; right:12px; font-size:10px; font-family:var(--font-mono); color:var(--cyan); background: rgba(34,211,238,0.1); padding:2px 7px; border-radius:99px; }

/* days grid */

.days-grid{ display:flex; flex-wrap:wrap; gap:8px; margin-bottom:20px; }

.day-chip{ width:64px; height:52px; border-radius:12px; border:1px solid var(--border); background: rgba(255,255,255,0.03); display:flex; align-items:center; justify-content:center; font-size:12.5px; font-weight:600; cursor:pointer; transition: all .15s; }

.day-chip.active{ background: var(--grad-primary); color:#04101f; border-color:transparent; }

/* toggle */

.toggle-row{ display:flex; align-items:center; justify-content:space-between; padding:14px 0; border-bottom:1px solid var(--border); }

.toggle-row:last-child{ border-bottom:none; }

.toggle-row .tr-label{ font-size:13.5px; font-weight:500; }

.toggle-row .tr-sub{ font-size:11.5px; color:var(--muted-dim); margin-top:2px; }

.switch{ position:relative; width:42px; height:24px; flex-shrink:0; }

.switch input{ opacity:0; width:0; height:0; }

.slider{ position:absolute; inset:0; background: rgba(255,255,255,0.12); border-radius:99px; cursor:pointer; transition: background .2s; }

.slider::before{ content:""; position:absolute; width:18px; height:18px; left:3px; top:3px; background:#fff; border-radius:50%; transition: transform .2s; }

input:checked + .slider{ background: var(--grad-primary); }

input:checked + .slider::before{ transform: translateX(18px); }

/* skip link */

```

```

.skip-line{ text-align:center; margin-top:18px; }

.skip-line button{ background:none; border:none; color:var(--muted-dim); font-size:12.5px;
cursor:pointer; text-decoration:underline; text-underline-offset:3px; }

.skip-line button:hover{ color:var(--muted); }


/* landing */

.landing-hero{ text-align:center; max-width:640px; margin: 6vh auto 0; padding: 0 16px; }

.landing-mark{ width:64px; height:64px; margin:0 auto 28px; }

.landing-tag{ font-family:var(--font-mono); font-size:12px; letter-spacing:0.2em; color:var(--cyan);
text-transform:uppercase; margin-bottom:18px; }

.landing-title{ font-family: var(--font-display); font-size: clamp(34px,6vw,58px); font-weight:700;
line-height:1.06; letter-spacing:-0.02em; margin-bottom:18px; }

.landing-title .accent{ background: var(--grad-primary); -webkit-background-clip:text; background-
clip:text; color:transparent; }

.landing-sub{ color: var(--muted); font-size:16px; line-height:1.6; max-width:46ch; margin:0 auto
34px; }

.landing-cta{ display:flex; gap:12px; justify-content:center; flex-wrap:wrap; }

.landing-loop{ display:flex; justify-content:center; gap:28px; margin-top:64px; flex-wrap:wrap; }

.loop-item{ text-align:center; font-family:var(--font-mono); font-size:11px; color:var(--muted-dim);
letter-spacing:0.08em; text-transform:uppercase; position:relative; padding: 0 6px;}

.loop-item b{ display:block; font-family:var(--font-display); font-size:20px; color:var(--text); font-
weight:600; letter-spacing:0; margin-bottom:4px; text-transform:none;}


/* preview card */

.preview-hero{ display:flex; gap:20px; align-items:center; margin-bottom:26px; flex-wrap:wrap; }

.preview-avatar{ width:76px; height:76px; border-radius:50%; background: var(--grad-primary);
display:flex; align-items:center; justify-content:center; font-family:var(--font-display); font-size:26px;
font-weight:700; color:#04101f; flex-shrink:0; overflow:hidden; }

.preview-avatar img{ width:100%; height:100%; object-fit:cover; }

.preview-name{ font-family:var(--font-display); font-size:21px; font-weight:600; }

.preview-username{ color:var(--cyan); font-family:var(--font-mono); font-size:13px; }

.preview-meta{ color:var(--muted); font-size:13px; margin-top:4px; }

.preview-section{ margin-bottom:22px; }

```

```

.preview-section-head{ display:flex; align-items:center; justify-content:between; justify-
content:space-between; margin-bottom:10px; }

.preview-section-head h3{ font-size:13px; font-weight:700; text-transform:uppercase; letter-
spacing:0.06em; color:var(--muted); }

.edit-link{ font-size:11.5px; color:var(--cyan); background:none; border:none; cursor:pointer; font-
family:var(--font-mono); }

.pill-row{ display:flex; flex-wrap:wrap; gap:7px; }

.pill{ font-size:12px; padding:6px 11px; border-radius:99px; border:1px solid var(--border);
background: rgba(255,255,255,0.03); }

.pill.teach{ border-color: rgba(59,130,246,0.4); color:#bcdcff; }

.pill.learn{ border-color: rgba(139,92,246,0.4); color:#e2d4ff; }

.bio-text{ font-size:13.5px; color:var(--muted); line-height:1.6; }

.completion-box{ display:flex; align-items:center; gap:16px; padding:18px; background:
rgba(59,130,246,0.06); border:1px solid rgba(59,130,246,0.25); border-radius: var(--radius-sm);
margin-bottom:24px; }

.ring{ width:56px; height:56px; flex-shrink:0; }

.completion-suggest{ font-size:12px; color:var(--muted-dim); margin-top:3px; }

/* success */

.success-wrap{ text-align:center; padding: 10px 0; }

.success-check{ width:76px; height:76px; margin:0 auto 22px; }

.success-actions{ display:grid; grid-template-columns:1fr 1fr 1fr; gap:12px; margin: 28px 0; }

@media(max-width:600px){ .success-actions{ grid-template-columns:1fr; } }

.success-action{ border:1px solid var(--border); border-radius: var(--radius-sm); padding:18px 12px;
cursor:pointer; background: rgba(255,255,255,0.03); transition: all .15s; }

.success-action:hover{ border-color: var(--border-bright); transform: translateY(-2px); }

.success-action .sa-icon{ font-size:22px; margin-bottom:8px; }

.success-action .sa-title{ font-size:13px; font-weight:600; }

/* dashboard */

.dash-header{ display:flex; align-items:center; justify-content:space-between; margin-bottom:28px;
flex-wrap:wrap; gap:14px;}

.dash-greeting h1{ font-family:var(--font-display); font-size: clamp(22px,3vw,28px); font-weight:600;
margin-bottom:4px;}

```

```

.dash-greeting p{ color:var(--muted); font-size:13.5px; }

.stat-grid{ display:grid; grid-template-columns: repeat(4,1fr); gap:14px; margin-bottom:30px; }

@media(max-width:720px){ .stat-grid{ grid-template-columns: repeat(2,1fr);} }

.stat-card{ background: var(--panel); border:1px solid var(--border); border-radius: var(--radius-sm);
padding:18px; }

.stat-card .num{ font-family:var(--font-display); font-size:28px; font-weight:700; background: var(--grad-primary); -webkit-background-clip:text; background-clip:text; color:transparent; }

.stat-card .lbl{ font-size:12px; color:var(--muted-dim); margin-top:4px; }

.section-head{ display:flex; align-items:center; justify-content:space-between; margin-bottom:16px;
}

.section-head h2{ font-family:var(--font-display); font-size:18px; font-weight:600; }

.match-grid{ display:grid; grid-template-columns: repeat(auto-fill,minmax(260px,1fr)); gap:16px; }

.match-card{ background: var(--panel); border:1px solid var(--border); border-radius: var(--radius);
padding:20px; transition: border-color .15s, transform .15s; }

.match-card:hover{ border-color: var(--border-bright); transform: translateY(-2px); }

.match-top{ display:flex; align-items:center; gap:12px; margin-bottom:14px; }

.match-avatar{ width:44px; height:44px; border-radius:50%; background: var(--grad-learn);
display:flex; align-items:center; justify-content:center; font-weight:700; font-family:var(--font-display); flex-shrink:0; }

.match-name{ font-weight:600; font-size:14.5px; }

.match-college{ font-size:11.5px; color:var(--muted-dim); }

.match-score{ margin-left:auto; text-align:right; }

.match-score b{ font-family:var(--font-display); font-size:17px; color:var(--good); display:block; }

.match-score span{ font-size:9.5px; color:var(--muted-dim); font-family:var(--font-mono); text-transform:uppercase; }

.match-row-label{ font-size:10.5px; color:var(--muted-dim); text-transform:uppercase; letter-spacing:0.05em; font-family:var(--font-mono); margin-bottom:5px; margin-top:10px;}

.match-actions{ display:flex; gap:8px; margin-top:16px; }

.match-actions .btn{ flex:1; font-size:12.5px; padding:10px; }

.perfect-tag{ display:inline-flex; align-items:center; gap:5px; font-size:11px; color:#fbbf24;
background: rgba(251,191,36,0.1); border:1px solid rgba(251,191,36,0.3); padding:4px 10px; border-radius:99px; margin-top:10px; }

.toast{ position:fixed; bottom:24px; left:50%; transform:translateX(-50%) translateY(20px);
background: #14192e; border:1px solid var(--border-bright); padding:13px 20px; border-radius:

```

```
12px; font-size:13px; box-shadow: 0 20px 40px rgba(0,0,0,0.4); z-index:100; opacity:0; transition: all
.3s; pointer-events:none; display:flex; align-items:center; gap:8px;}

.toast.show{ opacity:1; transform:translateX(-50%) translateY(0); }
```

```
@media(max-width:520px){ .actions-row{ flex-wrap:wrap; } }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="noise"></div>
```

```
<div id="app"></div>
```

```
<div class="toast" id="toast"></div>
```

```
<script>
```

```
/* ===== STATE ===== */
```

```
const SKILL_CATEGORIES = {
```

```
  "Technology": ["Web Development", "Java", "Python", "C++", "JavaScript", "App
Development", "UI/UX", "Git/GitHub"],
```

```
  "Creative": ["Photoshop", "Video Editing", "Photography", "Graphic Design", "Drawing", "Animation"],
```

```
  "Academic": ["Mathematics", "Physics", "Chemistry", "English"],
```

```
  "Other": ["Public Speaking", "Communication", "Digital Marketing", "Music", "Fitness"]
```

```
};
```

```
const TEACH_LEVELS = ["Beginner", "Intermediate", "Advanced", "Expert"];
```

```
const LEARN_LEVELS = ["Beginner", "Basic", "Intermediate"];
```

```
const DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
```

```
const TAKEN_USERNAMES = ["rahul", "priya", "admin", "test", "skillswap"];
```

```
let S = {
```

```
  step: 0,
```

```
  account: { fullName:"", username:"", email:"", phone:"", password:"", confirm:"", terms:false,
privacy:false },
```

```
  emailVerified: false,
```

```
  otpValue: ["", "", "", "", "", ""],
```

```

demoCode: "",
phoneVerified: false,
phoneSkipped: false,
phoneOtpValue: ["", "", "", "", "", ""],
phoneDemoCode: "",
phoneResendTimer: 30,
collegeVerified: "pending", // "verified" | "pending" | "manual"
collegeldText: "",
profile: { username:"", photo:null },
academic: { college:"", degree:"", branch:"", year:"", semester:"", gradYear:"", campusCity:"" },
about: { bio:"", languages:[] },
purpose: [],
teach: {}, // { skillName: level }
learn: {}, // { skillName: {level, reason} }
swapPref: "Skill for Skill",
partnerPref: { network:"Any college", course:"Any course" },
availability: { days:[], time:"Evening", customFrom:"", customTo:"", duration:"1 hour",
useCustom:false },
learningMode: [],
communication: [],
offlineLocation: "",
links: { github:"", linkedin:"", portfolio:"", behance:"" },
privacy: { visibility:"Campus Only", showEmail:false, showLinks:true, showAvailability:true },
trust: { completedSwaps: 0, ratingAvg: 0, responseRate: 96, cancellationRate: 2 },
reportedUsers: [],
blockedUsers: [],
errors: {}
};

```

```

const STEPS =
["landing", "signup", "verify", "phone", "profile", "academic", "collegeverify", "about", "purpose", "teach"

```

```
, "learn", "swap", "partner", "availability", "mode", "links", "privacy", "preview", "success", "dashboard", "admin"];
```

```
const ONBOARD_LABELS = {
```

```
  signup: "Create Account", verify: "Verify Email", phone: "Verify Phone", profile: "Basic Profile",  
  academic: "Academic Info",
```

```
  collegeverify: "College Verification", about: "About You", purpose: "Your Goals", teach: "Skills to  
  Teach", learn: "Skills to Learn", swap: "Swap Preference",
```

```
  partner: "Preferred Partner", availability: "Availability", mode: "Learning Mode & Contact",  
  links: "Portfolio", privacy: "Privacy", preview: "Preview"
```

```
};
```

```
const ONBOARD_SEQUENCE = Object.keys(ONBOARD_LABELS);
```

```
const EDU_DOMAIN_HINTS = [".edu", ".ac.in", ".edu.in", ".ac.uk", ".edu.au"];
```

```
function looksLikeCollegeEmail(email){
```

```
  const lower = (email || "").toLowerCase();
```

```
  return EDU_DOMAIN_HINTS.some(h=>lower.endsWith(h)) || /\.(edu|ac)\.[a-  
  z]{2,3}$/.test(lower.split('@')[1] || "");
```

```
}
```

```
function go(step){ S.step = STEPS.indexOf(step); S.errors = {}; render(); window.scrollTo({top:0,  
behavior:'smooth'})}; }
```

```
function toast(msg){
```

```
  const t = document.getElementById('toast');
```

```
  t.textContent = msg;
```

```
  t.classList.add('show');
```

```
  clearTimeout(window.__toastTimer);
```

```
  window.__toastTimer = setTimeout(()=>t.classList.remove('show'), 2600);
```

```
}
```

```
function esc(s){ const d=document.createElement('div'); d.textContent = s || ""; return d.innerHTML;  
}
```

```
/* ===== VALIDATION HELPERS ===== */
```

```
function pwScore(pw){
```

```
  let score = 0;
```

```

    if(pw.length>=8) score++;
    if(/[A-Z]/.test(pw)) score++;
    if(/[a-z]/.test(pw)) score++;
    if(/[0-9]/.test(pw)) score++;
    if(/[^\A-Za-z0-9]/.test(pw)) score++;
    return score;
}

function isValidEmail(e){ return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(e); }
function isValidUrl(u){ if(!u) return true; return /^https?:\/\/\.+\/.test(u); }

/* ===== RENDER ROOT ===== */
function render(){
    const step = STEPS[S.step];
    const app = document.getElementById('app');
    const isOnboarding = ONBOARD_SEQUENCE.includes(step);
    let html = "";

    if(step !== "landing" && step !== "success" && step !== "dashboard"){
        html += renderTopbar(isOnboarding, step);
    } else if(step === "dashboard"){
        html += renderTopbar(false, step, true);
    }

    html += `<main><div class="stage ${step==='dashboard'? 'wide':''}">`;

    switch(step){
        case "landing": html += renderLanding(); break;
        case "signup": html += renderSignup(); break;
        case "verify": html += renderVerify(); break;
        case "phone": html += renderPhone(); break;
        case "profile": html += renderProfile(); break;
    }

```

```

case "academic": html += renderAcademic(); break;
case "collegeverify": html += renderCollegeVerify(); break;
case "about": html += renderAbout(); break;
case "purpose": html += renderPurpose(); break;
case "teach": html += renderSkillPicker('teach'); break;
case "learn": html += renderSkillPicker('learn'); break;
case "swap": html += renderSwap(); break;
case "partner": html += renderPartner(); break;
case "availability": html += renderAvailability(); break;
case "mode": html += renderMode(); break;
case "links": html += renderLinks(); break;
case "privacy": html += renderPrivacy(); break;
case "preview": html += renderPreview(); break;
case "success": html += renderSuccess(); break;
case "dashboard": html += renderDashboard(); break;
case "admin": html += renderAdmin(); break;
}

```

```

html += `</div></main>`;
app.innerHTML = html;
afterRender(step);
}

```

```

function brandMarkSVG(size=30){
return `<svg viewBox="0 0 40 40" width="${size}" height="${size}">
<defs><linearGradient id="bg1" x1="0" y1="0" x2="1" y2="1">
  <stop offset="0" stop-color="#3b82f6"/><stop offset="1" stop-color="#22d3ee"/>
</linearGradient>
<linearGradient id="bg2" x1="0" y1="0" x2="1" y2="1">
  <stop offset="0" stop-color="#8b5cf6"/><stop offset="1" stop-color="#ec4899"/>
</linearGradient></defs>

```

```

    <circle cx="14" cy="14" r="10" fill="url(#bg1)" opacity="0.95"/>
    <circle cx="26" cy="26" r="10" fill="url(#bg2)" opacity="0.9"/>
  </svg>`;
}

function renderTopbar(showProgress, step, dashMode){
  let bar = `<div class="topbar">
    <div class="brand">${brandMarkSVG(28)}<span>SkillSwap<span style="color:var(--cyan)">Campus</span></span></div>
    ${dashMode ? `<div class="brand-tagline">LEARN • TEACH • SWAP • GROW</div>` :
    (showProgress ? `<button class="btn-text" style="font-size:12px" onclick="if(confirm('Exit onboarding and return to landing?')) go('landing')">Save & exit</button>` : '')}
  </div>`;
  if(showProgress){
    const idx = ONBOARD_SEQUENCE.indexOf(step);
    const pct = Math.round(((idx+1)/ONBOARD_SEQUENCE.length)*100);
    bar += `<div class="progress-shell">
      <div class="progress-track"><div class="progress-fill" style="width:${pct}%"></div></div>
      <div class="step-meta"><span>STEP ${idx+1} OF ${ONBOARD_SEQUENCE.length} —
      ${ONBOARD_LABELS[step].toUpperCase()}</span><span>${pct}%</span></div>
    </div>`;
  }
  return bar;
}

/* ===== LANDING ===== */
function renderLanding(){
  return `
  <div class="landing-hero">
    <div class="landing-mark">${brandMarkSVG(64)}</div>
    <div class="landing-tag">Learn • Teach • Swap • Grow</div>
    <h1 class="landing-title">Trade what you<br>know for what<br>you want to <span
    class="accent">learn</span></h1>

```

```
<p class="landing-sub">SkillSwap Campus connects students who can teach with students who want to learn — no tuition, just trade. Pick your skills, set your hours, meet your match.</p>
```

```
<div class="landing-cta">
```

```
<button class="btn btn-primary" onclick="go('signup')">Create your profile →</button>
```

```
<button class="btn btn-ghost" onclick="toast('This is a preview build — sign up to explore.')">See how it works</button>
```

```
</div>
```

```
<div class="landing-loop">
```

```
<div class="loop-item"><b>1,240+</b>students swapping</div>
```

```
<div class="loop-item"><b>38</b>skill categories</div>
```

```
<div class="loop-item"><b>92%</b>avg match score</div>
```

```
</div>
```

```
</div>`;
```

```
}
```

```
/* ===== SIGNUP ===== */
```

```
function renderSignup(){
```

```
  const a = S.account;
```

```
  const score = pwScore(a.password);
```

```
  const labels = ["Very weak", "Weak", "Fair", "Medium", "Strong", "Strong"];
```

```
  const colors = ["#f87171", "#f87171", "#fbbf24", "#fbbf24", "#34d399", "#34d399"];
```

```
  const e = S.errors;
```

```
  return `
```

```
<div class="card">
```

```
<div class="eyebrow">Create account</div>
```

```
<h1 class="step-title">Create your SkillSwap Campus account</h1>
```

```
<p class="step-sub">Join a community where students teach what they know and learn what they love.</p>
```

```
<div class="field-group">
```

```
<label class="field-label">Full name</label>
```

```

    <input type="text" id="f-fullname" class="${e.fullName?'err':''}" placeholder="e.g. Ananya
Sharma" value="${esc(a.fullName)}">

    ${e.fullName?'<div class="errmsg">⚠️ ${e.fullName}</div>':''}

</div>

<div class="field-group">

    <label class="field-label">Username</label>

    <input type="text" id="f-username" class="${e.username?'err':''}" placeholder="e.g.
ananya_codes" value="${esc(a.username)}">

    ${e.username?'<div class="errmsg">⚠️ ${e.username}</div>':''}

</div>

<div class="field-group">

    <label class="field-label">Email address</label>

    <input type="email" id="f-email" class="${e.email?'err':''}" placeholder="you@college.edu"
value="${esc(a.email)}">

    ${e.email?'<div class="errmsg">⚠️ ${e.email}</div>':''}

</div>

<div class="field-group">

    <label class="field-label">Password</label>

    <div class="pw-wrap">

        <input type="password" id="f-password" class="${e.password?'err':''}" placeholder="Create a
strong password" value="${esc(a.password)}">

        <button type="button" class="pw-toggle" onclick="togglePw('f-password')">SHOW</button>

    </div>

    <div class="pw-meter">${[0,1,2,3,4].map(i=><span
style="background:${score>i?colors[score]:''}"></span>`).join('')}</div>

    ${a.password ? `<div class="pw-strength-label"
style="color:${colors[score]}">${labels[score]}</div>` : ''}

    <div class="pw-reqs">

        ${reqRow(a.password.length>=8,"At least 8 characters")}

        ${reqRow(/[A-Z]/.test(a.password),"One uppercase letter")}

        ${reqRow(/[a-z]/.test(a.password),"One lowercase letter")}

        ${reqRow(/[0-9]/.test(a.password),"One number")}

        ${reqRow(/[^\A-Za-z0-9]/.test(a.password),"One special character")}

```

```

    </div>
</div>
<div class="field-group">
    <label class="field-label">Confirm password</label>
    <div class="pw-wrap">
        <input type="password" id="f-confirm" class="{e.confirm?'err':''}" placeholder="Re-enter your
password" value="{esc(a.confirm)}">
        <button type="button" class="pw-toggle" onclick="togglePw('f-confirm')">SHOW</button>
    </div>
    ${e.confirm?'<div class="errmsg">⚠ ${e.confirm}</div>':''}
</div>

    <label class="checkline"><input type="checkbox" id="f-terms" ${a.terms?'checked':''}><span>I
agree to the <a href="#" onclick="event.preventDefault()" style="color:var(--cyan)">Terms &
Conditions</a></span></label>

    <label class="checkline"><input type="checkbox" id="f-privacy" ${a.privacy?'checked':''}><span>I
agree to the <a href="#" onclick="event.preventDefault()" style="color:var(--cyan)">Privacy
Policy</a></span></label>

    ${e.consent?'<div class="errmsg">⚠ ${e.consent}</div>':''}

    <div class="actions-row" style="justify-content:flex-end">
        <button class="btn btn-primary btn-block" onclick="submitSignup()">Create Account
→</button>
    </div>
</div>;
}

function reqRow(met, label){
    return `<div class="pw-req ${met?'met':''}"><span class="dot">${met?'√':''}</span>${label}</div>`;
}

function togglePw(id){
    const el = document.getElementById(id);
    el.type = el.type === 'password' ? 'text' : 'password';
}

```

```

function submitSignup(){
  const a = S.account;

  a.fullName = val('f-fullname'); a.username = val('f-username'); a.email = val('f-email');

  a.password = val('f-password'); a.confirm = val('f-confirm');

  a.terms = document.getElementById('f-terms').checked;
  a.privacy = document.getElementById('f-privacy').checked;

  const e = {};

  if(!a.fullName.trim()) e.fullName = "Please enter your name.";
  if(!a.username.trim()) e.username = "Please choose a username.";

  else if(TAKEN_USERNAMES.includes(a.username.toLowerCase())) e.username = "Username already
taken.";

  if(!a.email.trim() || !isValidEmail(a.email)) e.email = "Please enter a valid email address.";
  if(pwScore(a.password) < 5) e.password = "Password must meet all requirements above.";
  if(a.password !== a.confirm) e.confirm = "Passwords do not match.";
  if(!a.terms || !a.privacy) e.consent = "Please accept the Terms and Privacy Policy to continue.";

  S.errors = e;

  if(Object.keys(e).length){ render(); return; }

  S.profile.username = a.username;

  // generate mock OTP (demo only — no real email backend in this prototype)

  S.demoCode = String(Math.floor(100000 + Math.random()*900000));

  S.otpValue = ['', '', '', '', '', ''];

  S.resendTimer = 30;

  toast("Account created — verification code sent.");

  go('verify');

  startResendTimer();
}

function val(id){ return document.getElementById(id) ? document.getElementById(id).value : ''; }

/* ===== VERIFY ===== */

function startResendTimer(){
  clearInterval(window.__resendInt);

```

```

window.__resendInt = setInterval(()=>{
  if(S.resendTimer>0){ S.resendTimer--; updateResendLabel(); }
  else clearInterval(window.__resendInt);
},1000);
}

function updateResendLabel(){
  const el = document.getElementById('resend-label');
  if(!el) return;

  if(S.resendTimer>0){ el.textContent = `Resend available in ${S.resendTimer}s`;
document.getElementById('resend-btn').disabled = true; }

  else { el.textContent = `Didn't get it?`; document.getElementById('resend-btn').disabled = false; }
}

function renderVerify(){
  return `
<div class="card">

  <div class="eyebrow">Verify email</div>

  <h1 class="step-title">Verify your email</h1>

  <p class="step-sub">We've sent a verification code to <strong style="color:var(--
text)">${esc(S.account.email)}</strong>.</p>

  <div class="demo-code-banner">Prototype mode — no real email is sent. Your demo code is
<strong>${S.demoCode}</strong></div>

  <div class="otp-row">

    ${[[0,1,2,3,4,5].map(i=>`<input maxlength="1" inputmode="numeric" id="otp-${i}"
value="${S.otpValue[i]}" oninput="otpInput(${i}, this)">`).join("")}

  </div>

  ${S.errors.otp ? `<div class="errmsg" style="justify-content:center">⚠️ ${S.errors.otp}</div>`.`"}

  <div class="resend-line">

    <span id="resend-label">Resend available in ${S.resendTimer} | 30s</span>

    <button id="resend-btn" onclick="resendOtp()" ${S.resendTimer>0?'disabled':''}>Resend
Code</button>

  </div>

  <div class="actions-row">

    <button class="btn btn-ghost" onclick="go('signup')">← Back</button>

```

```

        <button class="btn btn-primary" onclick="verifyOtp()">Verify Email</button>
    </div>
</div>`;
}
function otpInput(i, el){
    el.value = el.value.replace(/[^0-9]/g,"").slice(0,1);
    S.otpValue[i] = el.value;
    if(el.value && i<5){ document.getElementById(`otp-${i+1}`).focus(); }
}
function resendOtp(){
    S.demoCode = String(Math.floor(100000 + Math.random()*900000));
    S.resendTimer = 30;
    toast("New code sent.");
    render();
    startResendTimer();
}
function verifyOtp(){
    const code = S.otpValue.join("");
    if(code.length<6){ S.errors = {otp:"Please enter all 6 digits."}; render(); return; }
    if(code !== S.demoCode){ S.errors = {otp:"That code doesn't match. Check and try again."};
    render(); return; }
    S.emailVerified = true;
    toast("Email verified ✓");
    S.phoneDemoCode = String(Math.floor(100000 + Math.random()*900000));
    S.phoneResendTimer = 30;
    go('phone');
    startPhoneResendTimer();
}

/* ===== PHONE VERIFICATION ===== */
function startPhoneResendTimer(){

```

```

clearInterval(window.__phoneResendInt);

window.__phoneResendInt = setInterval(()=>{

  if(S.phoneResendTimer>0){ S.phoneResendTimer--; updatePhoneResendLabel(); }

  else clearInterval(window.__phoneResendInt);

},1000);

}

function updatePhoneResendLabel(){

  const el = document.getElementById('phone-resend-label');

  if(!el) return;

  if(S.phoneResendTimer>0){ el.textContent = `Resend available in ${S.phoneResendTimer}s`;
document.getElementById('phone-resend-btn').disabled = true; }

  else { el.textContent = `Didn't get it?`; document.getElementById('phone-resend-btn').disabled =
false; }

}

function renderPhone(){

  const sentState = S.account.__phoneSent;

  return `

<div class="card">

  <div class="eyebrow">Phone verification <span class="opt" style="color:var(--muted-dim); font-
weight:400">optional but recommended</span></div>

  <h1 class="step-title">Verify your phone number</h1>

  <p class="step-sub">A verified phone cuts down on fake accounts and helps partners reach you if
plans change. You can skip this and add it later.</p>

  ${!sentState ? `

  <div class="field-group">

    <label class="field-label">Mobile number</label>

    <input type="text" id="f-phone" placeholder="+91 98765 43210"
value="${esc(S.account.phone)}">

    ${S.errors.phone ? `<div class="errmsg">⚠️ ${S.errors.phone}</div>:`:''}

  </div>

  <div class="actions-row">

    <button class="btn btn-ghost" onclick="go('verify')">⬅️ Back</button>

```

```

<div class="left-actions">

  <button class="btn-text" onclick="S.phoneSkipped=true; go('profile');">Skip for now</button>

  <button class="btn btn-primary" onclick="sendPhoneOtp()">Send Code →</button>

</div>

</div>

` : `

<p class="step-sub" style="margin-bottom:14px">Code sent to <strong style="color:var(--text)">${esc(S.account.phone)}</strong>.</p>

<div class="demo-code-banner">Prototype mode — no real SMS is sent. Your demo code is
<strong>${S.phoneDemoCode}</strong></div>

<div class="otp-row">

  ${[0,1,2,3,4,5].map(i=>`<input maxlength="1" inputmode="numeric" id="potp-${i}"
value="${S.phoneOtpValue[i]}" oninput="phoneOtpInput(${i}, this)">`).join("")}

</div>

${S.errors.otp ? `<div class="errmsg" style="justify-content:center">⚠ ${S.errors.otp}</div>`:''}

<div class="resend-line">

  <span id="phone-resend-label">Resend available in ${S.phoneResendTimer}s</span>

  <button id="phone-resend-btn" onclick="resendPhoneOtp()"
${S.phoneResendTimer>0?'disabled':''}>Resend Code</button>

</div>

<div class="actions-row">

  <button class="btn btn-ghost" onclick="S.account.__phoneSent=false; render();">← Change
number</button>

  <div class="left-actions">

    <button class="btn-text" onclick="S.phoneSkipped=true; go('profile');">Skip for now</button>

    <button class="btn btn-primary" onclick="verifyPhoneOtp()">Verify Phone</button>

  </div>

</div>

` }

</div>`;

}

function sendPhoneOtp(){

  const phone = val('f-phone').trim();

```

```

    if(!phone || phone.replace(/\D/g,"").length<7){ S.errors={phone:"Please enter a valid phone
number."}; render(); return; }

    S.account.phone = phone;

    S.account.__phoneSent = true;

    S.phoneOtpValue = ["", "", "", "", "", ""];

    S.errors = {};

    render();

    document.getElementById('potp-0')?.focus();
}

function phoneOtpInput(i, el){
    el.value = el.value.replace(/[^0-9]/g,"").slice(0,1);

    S.phoneOtpValue[i] = el.value;

    if(el.value && i<5){ document.getElementById(`potp-${i+1}`).focus(); }
}

function resendPhoneOtp(){
    S.phoneDemoCode = String(Math.floor(100000 + Math.random()*900000));

    S.phoneResendTimer = 30;

    toast("New code sent.");

    render();

    startPhoneResendTimer();
}

function verifyPhoneOtp(){
    const code = S.phoneOtpValue.join("");

    if(code.length<6){ S.errors={otp:"Please enter all 6 digits."}; render(); return; }

    if(code !== S.phoneDemoCode){ S.errors={otp:"That code doesn't match. Check and try again."};
render(); return; }

    S.phoneVerified = true;

    toast("Phone verified ✓");

    go('profile');
}

/* ===== BASIC PROFILE ===== */

```

```

function renderProfile(){
  const p = S.profile;
  const e = S.errors;
  let availTag = '';
  if(p.username && p.username !== S.account.username){
    availTag = TAKEN_USERNAMES.includes(p.username.toLowerCase())
      ? `<div class="avail-tag taken">× Username already taken</div>`
      : `<div class="avail-tag ok">✓ Available</div>`;
  } else if(p.username){
    availTag = `<div class="avail-tag ok">✓ Available</div>`;
  }
  return `
<div class="card">
  <div class="eyebrow">Basic profile</div>
  <h1 class="step-title">Let's build your profile</h1>
  <p class="step-sub">This is how other students on campus will find and recognize you.</p>

  <div class="field-group">
    <label class="field-label">Full name</label>
    <input type="text" value="${esc(S.account.fullName)}" disabled style="opacity:0.6">
  </div>

  <div class="field-group">
    <label class="field-label">Username</label>
    <input type="text" id="f-username2" class="${e.username?'err':''}" value="${esc(p.username)}"
oninput="checkUsername(this.value)">
    ${availTag}
    ${e.username?'<div class="errmsg">⚠️ ${e.username}</div>':''}
  </div>

  <div class="field-group">
    <label class="field-label">Profile photo <span class="opt">optional</span></label>

```

```

<div class="photo-row">

  <div class="avatar-circle ${p.photo?'filled':''}" id="avatar-preview">${p.photo ? `` : 'No photo'}</div>

  <div>

    <input type="file" accept="image/*" id="photo-input" style="display:none"
onchange="handlePhoto(this)">

    <button class="btn btn-ghost" onclick="document.getElementById('photo-
input').click()">Upload photo</button>

  </div>

</div>

```

```

<div class="actions-row">

  <button class="btn btn-ghost" onclick="go('phone')">← Back</button>

  <div class="left-actions">

    <button class="btn-text" onclick="go('academic')">Skip for now</button>

    <button class="btn btn-primary" onclick="submitProfile()">Continue →</button>

  </div>

</div>
</div>;

```

```

}

```

```

function checkUsername(v){ S.profile.username = v; render(); document.getElementById('f-
username2').focus(); document.getElementById('f-
username2').setSelectionRange(v.length,v.length); }

```

```

function handlePhoto(input){

  const file = input.files[0];

  if(!file) return;

  const reader = new FileReader();

  reader.onload = e => { S.profile.photo = e.target.result; render(); };

  reader.readAsDataURL(file);

}

```

```

function submitProfile(){

  const u = S.profile.username;

```

```

if(!u.trim()){ S.errors = {username:"Please choose a username."}; render(); return; }

if(TAKEN_USERNAMES.includes(u.toLowerCase()) && u !== S.account.username){ S.errors =
{username:"That username is taken — try another."}; render(); return; }

go('academic');
}

/* ===== ACADEMIC ===== */

function renderAcademic(){
  const a = S.academic; const e = S.errors;

  const degrees = ["B.Tech","BCA","MCA","BBA","MBA","BA","B.Sc","M.Sc","Diploma","Other"];
  const years = ["1st Year","2nd Year","3rd Year","4th Year","Postgraduate","Other"];
  const gradYears = Array.from({length:7},(_i)=>2026+i);

  return `
<div class="card">

  <div class="eyebrow">Academic info</div>

  <h1 class="step-title">Tell us about your campus life</h1>

  <p class="step-sub">This helps us match you with students on similar academic tracks.</p>


  <div class="field-group">

    <label class="field-label">College / University</label>

    <input type="text" id="f-college" class="${e.college?'err':''}" placeholder="e.g. IIT Patna"
value="${esc(a.college)}">

    ${e.college?'<div class="errmsg">⚠️ ${e.college}</div>':''}

  </div>

  <div class="grid2">

    <div class="field-group">

      <label class="field-label">Course / Degree</label>

      <select id="f-degree">${["Select...",...degrees].map(d=>`<option ${a.degree===d?'selected':''}
${d==='Select...'?'disabled':''}
${(!a.degree&&d==='Select...')?'selected':''}>${d}</option>`).join("")}</select>

    </div>

    <div class="field-group">

```

```

    <label class="field-label">Branch / Specialization</label>

    <input type="text" id="f-branch" placeholder="e.g. Computer Science"
value="\${esc(a.branch)}">

    </div>

</div>

<div class="grid2">

    <div class="field-group">

        <label class="field-label">Current year</label>

        <select id="f-year">\${["Select...",...years].map(y=>\`<option \${a.year===y?'selected':''}
\${y==='Select...'?'disabled':''}
\${(!a.year&&y==='Select...')?'selected':''}>\${y}</option>\`}).join("")</select>

        </div>

        <div class="field-group">

            <label class="field-label">Semester <span class="opt">optional</span></label>

            <input type="text" id="f-semester" placeholder="e.g. 5" value="\${esc(a.semester)}">

            </div>

        </div>

        <div class="field-group">

            <label class="field-label">Graduation year</label>

            <select id="f-gradyear">\${["Select...",...gradYears].map(y=>\`<option
\${String(a.gradYear)===String(y)?'selected':''} \${y==='Select...'?'disabled':''}
\${(!a.gradYear&&y==='Select...')?'selected':''}>\${y}</option>\`}).join("")</select>

            </div>

        <div class="field-group">

            <label class="field-label">Campus / City</label>

            <input type="text" id="f-campusCity" class="\${e.campusCity?'err':''}" placeholder="e.g. IIT Patna
campus, or Patna" value="\${esc(a.campusCity)}">

            <div class="hint">Used to match you with nearby students for offline swaps — never your exact
address.</div>

            \${e.campusCity?\`<div class="errmsg">\u26a0 \${e.campusCity}</div>`:''}

        </div>

    \${e.form?\`<div class="errmsg">\u26a0 \${e.form}</div>`:''}

```

```

<div class="actions-row">

  <button class="btn btn-ghost" onclick="go('profile')">← Back</button>

  <button class="btn btn-primary" onclick="submitAcademic()">Continue →</button>

</div>

</div>`;
}

function submitAcademic(){
  const a = S.academic;

  a.college = val('f-college'); a.degree = val('f-degree'); a.branch = val('f-branch');
  a.year = val('f-year'); a.semester = val('f-semester'); a.gradYear = val('f-gradyear');
  a.campusCity = val('f-campuscity');

  if(!a.college.trim()){ S.errors = {college:"Please enter your college or university."}; render(); return; }

  if(a.degree==="Select..." || a.year==="Select..."){ S.errors = {form:"Please select your course and
current year."}; render(); return; }

  if(!a.campusCity.trim()){ S.errors = {campusCity:"Please enter your campus or city."}; render();
return; }

  // Auto-check college verification based on email domain, then move on
  S.collegeVerified = looksLikeCollegeEmail(S.account.email) ? "verified" : "pending";

  go('collegeverify');
}

/* ===== COLLEGE VERIFICATION =====
*/

function renderCollegeVerify(){
  const status = S.collegeVerified;

  const emailLooksEdu = looksLikeCollegeEmail(S.account.email);

  return `
<div class="card">

  <div class="eyebrow">College verification</div>

  <h1 class="step-title">Let's verify your student status</h1>

  <p class="step-sub">A verified badge builds trust with other students and improves your match
quality.</p>

```

```

    ${status===`verified` ? `

```

```

        <div class="toggle-row" style="border:1px solid rgba(52,211,153,0.35);
background:rgba(52,211,153,0.08); border-radius:var(--radius-sm); padding:16px; margin-
bottom:20px">

```

```

            <div><div class="tr-label" style="color:var(--good)">✓ Verified Student</div><div class="tr-
sub">${esc(S.account.email)} matches a recognized college email pattern.</div></div>

```

```

        </div>

```

```

    ` : `

```

```

        <div class="toggle-row" style="border:1px solid var(--border);
background:rgba(255,255,255,0.03); border-radius:var(--radius-sm); padding:16px; margin-
bottom:20px">

```

```

            <div><div class="tr-label" style="color:var(--warn)">⌚ Verification pending</div><div class="tr-
sub">${emailLooksEdu ? "We couldn't confirm this automatically yet." : "Your email doesn't look like
a college-issued address. Add a college ID number below, or switch to your .edu / institute email
later from settings."}</div></div>

```

```

        </div>

```

```

        <div class="field-group">

```

```

            <label class="field-label">College ID number <span class="opt">optional, for manual
review</span></label>

```

```

            <input type="text" id="f-collegeid" placeholder="e.g. 2024CSB1042"
value="${esc(S.collegelIdText)}" oninput="S.collegelIdText=this.value">

```

```

            <div class="hint">Submitted IDs are reviewed manually — this is simulated in this
prototype.</div>

```

```

        </div>

```

```

    `}

```

```

<div class="actions-row">

```

```

    <button class="btn btn-ghost" onclick="go('academic')">← Back</button>

```

```

    <div class="left-actions">

```

```

        ${status!=='verified' ? `<button class="btn-text" onclick="go('about')">Verify later</button>` : ``}

```

```

        <button class="btn btn-primary" onclick="submitCollegeVerify()">Continue →</button>

```

```

    </div>

```

```

</div>

```

```

</div>;

```

```

}

function submitCollegeVerify(){

  if(S.collegeVerified!=='verified' && S.collegeIdText.trim()){ S.collegeVerified = 'manual';
  toast('College ID submitted for review.');
```

go('about');

```

}

/* ===== ABOUT ===== */

function renderAbout(){

  const a = S.about; const e = S.errors;

  const langs = ["English","Hindi","Bengali","Tamil","Telugu","Marathi","Other"];

  return `

<div class="card">

  <div class="eyebrow">About you</div>

  <h1 class="step-title">Tell the community about yourself</h1>

  <p class="step-sub">A short, honest bio helps future skill partners know who they're talking
to.</p>

  <div class="field-group">

    <label class="field-label">Bio</label>

    <textarea id="f-bio" class="${e.bio?'err':''}" maxlength="260" placeholder="Example: I'm a B.Tech
CSE student interested in web development, UI design and photography. I enjoy learning new
technologies and helping other students." oninput="updateBioCount(this)">${esc(a.bio)}</textarea>

    <div class="charcount" id="bio-count">${a.bio.length} / 150–250 characters</div>

    ${e.bio?'<div class="errmsg">⚠️ ${e.bio}</div>':''}

  </div>

  <div class="field-group">

    <label class="field-label">Languages you speak</label>

    <div class="chip-field">

      ${langs.map(l=>`<div class="chip ${a.languages.includes(l)?'active':''}"
onclick="toggleLang('${l}')">${l}</div>`).join("")}

    </div>

```

```
</div>
```

```
<div class="actions-row">
```

```
  <button class="btn btn-ghost" onclick="go('collegeverify')">← Back</button>
```

```
  <button class="btn btn-primary" onclick="submitAbout()">Continue →</button>
```

```
</div>
```

```
</div>`;
```

```
}
```

```
function updateBioCount(el){
```

```
  const n = el.value.length;
```

```
  const c = document.getElementById('bio-count');
```

```
  c.textContent = `${n} / 150–250 characters`;
```

```
  c.className = 'charcount' + (n>250?' over:');
```

```
}
```

```
function toggleLang(l){
```

```
  const i = S.about.languages.indexOf(l);
```

```
  if(i>-1) S.about.languages.splice(i,1); else S.about.languages.push(l);
```

```
  render();
```

```
}
```

```
function submitAbout(){
```

```
  const bio = val('f-bio');
```

```
  S.about.bio = bio;
```

```
  if(bio.trim().length < 150 || bio.trim().length > 250){ S.errors = {bio:"Your bio should be between 150–250 characters."}; render(); return; }
```

```
  go('purpose');
```

```
}
```

```
/* ===== PURPOSE / GOALS ===== */
```

```
function renderPurpose(){
```

```
  const goals = ["Learn new skills","Teach my skills","Find project partners","Build portfolio","Meet other students","Prepare for career","Academic help"];
```

```
  const err = S.errors.purpose;
```

```

return `
<div class="card">

  <div class="eyebrow">Your goals</div>

  <h1 class="step-title">Why are you joining SkillSwap?</h1>

  <p class="step-sub">Pick everything that applies — this shapes the partners and skills we
recommend.</p>


  <div class="chip-field">

    ${goals.map(g=><div class="chip ${S.purpose.includes(g)?'active':''}"
onclick="togglePurpose('${g.replace(/'/g, '\\\'')}')">${g}</div>`).join("")}

  </div>

  ${err?<div class="errmsg">⚠️ ${err}</div>:''}


  <div class="actions-row">

    <button class="btn btn-ghost" onclick="go('about')">⬅️ Back</button>

    <button class="btn btn-primary" onclick="submitPurpose()">Continue ➡️</button>

  </div>
</div>;
}

function togglePurpose(g){
  const i = S.purpose.indexOf(g);
  if(i>-1) S.purpose.splice(i,1); else S.purpose.push(g);
  render();
}

function submitPurpose(){
  if(S.purpose.length===0){ S.errors={purpose:"Please select at least one reason for joining."};
render(); return; }

  go('teach');
}

/* ===== SKILL PICKER (teach / learn)
===== */

```

```

function renderSkillPicker(mode){
  const isTeach = mode==='teach';
  const selected = isTeach ? S.teach : S.learn;
  const selectedNames = Object.keys(selected);
  const query = S.skillQuery || '';
  const levels = isTeach ? TEACH_LEVELS : LEARN_LEVELS;

  let catHtml = '';

  Object.entries(SKILL_CATEGORIES).forEach(([cat, skills])=>{
    const filtered = skills.filter(s=>s.toLowerCase().includes(query.toLowerCase()));
    if(!filtered.length) return;
    catHtml += `<div class="skill-cat"><div class="skill-cat-label">${cat}</div><div class="skill-grid">
      ${filtered.map(s=>`<div class="skill-chip ${mode} ${selected[s]?'active':''}"
onclick="toggleSkill('${mode}', '${s.replace(/'/g, '\\')}')">${s}</div>`).join("")}
    </div></div>`;
  });
}

```

```

let selectedHtml = '';
if(selectedNames.length===0){
  selectedHtml = `<div class="empty-note">No skills selected yet — tap a skill above to add
it.</div>`;
} else {
  selectedHtml = selectedNames.map(name=>{
    const entry = selected[name];
    const currentLevel = isTeach ? entry : entry.level;
    const reason = isTeach ? null : entry.reason;
    return `<div class="selected-item">
      <div style="flex:1">
        <div class="name">${esc(name)}</div>
        <div class="level-select" style="margin-top:8px">
          ${levels.map(lv=>`<button class="level-btn ${currentLevel===lv?'active':''}"
onclick="setSkillLevel('${mode}', '${name.replace(/'/g, '\\')}', '${lv}')">${lv}</button>`).join("")}
        </div>
      </div>`;
  });
}

```

```

    </div>

    ${!isTeach ? `<input type="text" class="reason-input" placeholder="Why do you want to learn
this? (optional)" value="${esc(reason | | '"')}" oninput="setLearnReason('${name.replace(/'/g, "\\')}',
this.value)">` : ""}

    </div>

    <button class="remove-x"
onclick="toggleSkill('${mode}', '${name.replace(/'/g, "\\')}')">X</button>

</div>`;
}).join("");
}

```

```

const title = isTeach ? "What can you teach?" : "What do you want to learn?";

const sub = isTeach ? "Choose the skills you're confident enough to share with another student." :
"Choose the skills you want to learn from other students.";

const backStep = isTeach ? "purpose" : "teach";

const nextStep = isTeach ? "learn" : "swap";

const err = S.errors.skills;

return `
<div class="card">

  <div class="eyebrow">${isTeach?'Teaching skills':'Learning goals'}</div>

  <h1 class="step-title">${title}</h1>

  <p class="step-sub">${sub}</p>

  <div class="search-box"><input type="text" placeholder="Search skills..." value="${esc(query)}"
oninput="S.skillQuery=this.value; render();"></div>

  ${catHtml | | `<div class="empty-note">No skills match "${esc(query)}" — try adding it as a custom
skill below.</div>`}

  <div class="add-custom">

    <input type="text" id="custom-skill-input" placeholder="+ Add a custom skill">

    <button class="btn btn-ghost" onclick="addCustomSkill('${mode}')">Add</button>

  </div>

```

```

<div class="selected-panel">

  <div class="selected-panel-title"><span>Selected</span><span
class="count">${selectedNames.length} skill${selectedNames.length!==1?'s':''}</span></div>

  ${selectedHtml}

</div>

${err?`<div class="errmsg">⚠ ${err}</div>`:''}

<div class="actions-row">

  <button class="btn btn-ghost" onclick="go('${backStep}')">← Back</button>

  <button class="btn btn-primary" onclick="submitSkills('${mode}','${nextStep}')">Continue
→</button>

</div>

</div>;
}

function toggleSkill(mode, name){
  const store = mode==='teach' ? S.teach : S.learn;
  if(store[name]) delete store[name];
  else store[name] = mode==='teach' ? TEACH_LEVELS[1] : {level: LEARN_LEVELS[0], reason:''};
  render();
}

function setSkillLevel(mode, name, level){
  if(mode==='teach') S.teach[name] = level;
  else S.learn[name].level = level;
  render();
}

function setLearnReason(name, reason){ if(S.learn[name]) S.learn[name].reason = reason; }

function addCustomSkill(mode){
  const input = document.getElementById('custom-skill-input');
  const name = input.value.trim();
  if(!name) return;
  const store = mode==='teach' ? S.teach : S.learn;

```

```

store[name] = mode==='teach' ? TEACH_LEVELS[1] : {level: LEARN_LEVELS[0], reason:""};
S.skillQuery = "";
render();
}

function submitSkills(mode, nextStep){
  const store = mode==='teach' ? S.teach : S.learn;
  if(Object.keys(store).length===0){
    S.errors = {skills: mode==='teach' ? "Please select at least one skill you can teach." : "Please select
at least one skill you want to learn."};
    render(); return;
  }
  S.skillQuery = "";
  go(nextStep);
}

```

```

/* ===== SWAP PREFERENCE ===== */

```

```

function renderSwap(){
  const options = [
    {v:"Skill for Skill", d:"Trade your teaching for someone else's — the core SkillSwap model."},
    {v:"Teach Only", d:"You're here mainly to share what you know."},
    {v:"Learn Only", d:"You're here mainly to pick up new skills."},
    {v:"Open to Everything", d:"Flexible — match me however works best."}
  ];
  return `
<div class="card">
  <div class="eyebrow">Swap preference</div>
  <h1 class="step-title">How do you want to exchange skills?</h1>
  <p class="step-sub">SkillSwap works by connecting students who can help each other.</p>

  <div class="swap-diagram">
    <div class="swap-node a"><div class="badge">📚</div>You teach<br>JavaScript</div>

```

```
<div class="swap-arrows">↔</div>
```

```
<div class="swap-node b"><div class="badge">🔖</div>They teach<br>UI/UX</div>
</div>
```

```
<div class="radio-cards">
```

```
  ${options.map(o=>`<div class="radio-card ${S.swapPref===o.v?'active':''}"
  onclick="S.swapPref='${o.v}'; render();">
```

```
    ${o.v==='Skill for Skill'?<div class="rc-badge">Recommended</div>:''}
```

```
    <div class="rc-title">${o.v}</div>
```

```
    <div class="rc-sub">${o.d}</div>
```

```
  </div>`).join(")"}
</div>
```

```
<div class="actions-row">
```

```
  <button class="btn btn-ghost" onclick="go('learn')">← Back</button>
```

```
  <button class="btn btn-primary" onclick="go('partner')">Continue →</button>
```

```
</div>
```

```
</div>`;
```

```
}
```

```
/* ===== PREFERRED PARTNER ===== */
```

```
function renderPartner(){
```

```
  const p = S.partnerPref;
```

```
  return `
```

```
    <div class="card">
```

```
      <div class="eyebrow">Preferred partner</div>
```

```
      <h1 class="step-title">Who would you like to learn with?</h1>
```

```
      <p class="step-sub">This fine-tunes your matches — you can widen it any time.</p>
```

```
    <div class="field-group">
```

```
      <label class="field-label">College</label>
```

```

    <div class="chip-field">
      ${["Same college", "Any college"].map(v=><div class="chip ${p.network===v?'active':''}"
onclick="S.partnerPref.network='${v}'; render();">${v}</div>`).join("")}
    </div>
  </div>

  <div class="field-group">
    <label class="field-label">Course</label>

    <div class="chip-field">
      ${["Same course", "Any course"].map(v=><div class="chip ${p.course===v?'active':''}"
onclick="S.partnerPref.course='${v}'; render();">${v}</div>`).join("")}
    </div>
  </div>

  <div class="actions-row">
    <button class="btn btn-ghost" onclick="go('swap')">← Back</button>
    <button class="btn btn-primary" onclick="go('availability')">Continue →</button>
  </div>
</div>;
}

/* ===== AVAILABILITY ===== */
function renderAvailability(){
  const av = S.availability;
  const times = ["Morning", "Afternoon", "Evening", "Night"];
  const durations = ["30 minutes", "45 minutes", "1 hour", "2 hours"];
  return `
  <div class="card">
    <div class="eyebrow">Availability</div>
    <h1 class="step-title">When are you available?</h1>
    <p class="step-sub">Pick the days and times that usually work for you — you can always change
this later.</p>

```

```

<div class="field-group">

  <label class="field-label">Available days</label>

  <div class="days-grid">

    ${DAYS.map(d=><div class="day-chip ${av.days.includes(d)?'active':''}"
onclick="toggleDay('${d}')">${d}</div>`).join("")}

    </div>

  </div>

<div class="field-group">

  <label class="field-label">Preferred time</label>

  <div class="chip-field" style="margin-bottom:14px">

    ${times.map(t=><div class="chip ${!av.useCustom && av.time===t?'active':''}"
onclick="setPreferredTime('${t}')">${t}</div>`).join("")}

    <div class="chip ${av.useCustom?'active':''}" onclick="av_setCustom()">Custom time</div>

  </div>

  ${av.useCustom ? `

  <div class="grid2">

    <div><label class="field-label">From</label><input type="time" id="f-from"
value="${av.customFrom | '18:00'}" onchange="S.availability.customFrom=this.value"></div>

    <div><label class="field-label">To</label><input type="time" id="f-to"
value="${av.customTo | '21:00'}" onchange="S.availability.customTo=this.value"></div>

  </div>` : ""}

  </div>

<div class="field-group">

  <label class="field-label">Session duration</label>

  <div class="chip-field">

    ${durations.map(d=><div class="chip ${av.duration===d?'active':''}"
onclick="S.availability.duration='${d}'; render();">${d}</div>`).join("")}

    </div>

  </div>

  ${S.errors.days?`<div class="errmsg">⚠ ${S.errors.days}</div>`: ""}

```

```

<div class="actions-row">

  <button class="btn btn-ghost" onclick="go('partner')">← Back</button>

  <button class="btn btn-primary" onclick="submitAvailability()">Continue →</button>

</div>

</div>;
}

function toggleDay(d){
  const i = S.availability.days.indexOf(d);
  if(i>-1) S.availability.days.splice(i,1); else S.availability.days.push(d);
  render();
}

function setPreferredTime(t){ S.availability.useCustom=false; S.availability.time=t; render(); }

function av_setCustom(){ S.availability.useCustom = true; render(); }

function submitAvailability(){
  if(S.availability.days.length===0){ S.errors={days:"Please select at least one available day."};
  render(); return; }

  go('mode');
}

/* ===== LEARNING MODE ===== */

function renderMode(){
  const modes = [
    {v:"Online", d:"Video call, chat, or screen sharing.", icon:"💻"},
    {v:"Offline", d:"Meet in person on campus.", icon:"📍"},
    {v:"Both", d:"Open to either, whatever suits the swap.", icon:"🔄"}
  ];
  return `
<div class="card">

  <div class="eyebrow">Learning mode</div>

  <h1 class="step-title">How do you prefer to learn?</h1>

  <p class="step-sub">Select all that apply.</p>

```

```

<div class="radio-cards" style="grid-template-columns:1fr">

  ${modes.map(m=>`<div class="radio-card ${S.learningMode.includes(m.v)?'active':''}"
onclick="toggleMode('${m.v}')">

    <div class="rc-title">${m.icon} ${m.v}</div><div class="rc-sub">${m.d}</div>

  </div>`).join("")}

</div>

${S.learningMode.includes('Offline')} ? `

<div class="field-group" style="margin-top:20px">

  <label class="field-label">Preferred campus / location</label>

  <input type="text" id="f-location" placeholder="e.g. Central Library, IIT Patna"
value="${esc(S.offlineLocation)}" oninput="S.offlineLocation=this.value">

  <div class="hint">We only ask for a general meeting spot — never your home address.</div>

</div>` : ""

<div class="field-group" style="margin-top:24px">

  <label class="field-label">Preferred communication</label>

  <div class="chip-field">

    ${["Chat","Voice Call","Video Call","In-person"].map(c=>`<div class="chip
${S.communication.includes(c)?'active':''}" onclick="toggleComm('${c}')">${c}</div>`).join("")}

  </div>

  ${S.errors.comm?`<div class="errmsg">⚠️ ${S.errors.comm}</div>`:''}

</div>

${S.errors.mode?`<div class="errmsg">⚠️ ${S.errors.mode}</div>`:''}

<div class="actions-row">

  <button class="btn btn-ghost" onclick="go('availability')">⬅️ Back</button>

  <button class="btn btn-primary" onclick="submitMode()">Continue ➡️</button>

</div>

</div>;

}

```

```

function toggleMode(v){
  const i = S.learningMode.indexOf(v);
  if(i>-1) S.learningMode.splice(i,1); else S.learningMode.push(v);
  render();
}

function toggleComm(c){
  const i = S.communication.indexOf(c);
  if(i>-1) S.communication.splice(i,1); else S.communication.push(c);
  render();
}

function submitMode(){
  if(S.learningMode.length===0){ S.errors={mode:"Please select at least one learning mode."};
  render(); return; }

  if(S.communication.length===0){ S.errors={comm:"Please select at least one way to
  communicate."}; render(); return; }

  go('links');
}

/* ===== LINKS ===== */

function renderLinks(){
  const l = S.links; const e = S.errors;

  const fields =
  [{"github","GitHub","https://github.com/yourname"}, {"linkedin","LinkedIn","https://linkedin.com/in/yourname"}, {"portfolio","Portfolio website","https://yourname.dev"}, {"behance","Behance","https://behance.net/yourname"}];

  return `
  <div class="card">
    <div class="eyebrow">Portfolio · optional</div>
    <h1 class="step-title">Showcase your work</h1>
    <p class="step-sub">These links appear on your public profile only if you add them.</p>

    ${fields.map(([key,label,ph])=>`
      <div class="field-group">

```

```

    <label class="field-label">${label} <span class="opt">optional</span></label>

    <input type="url" id="f-${key}" class="${e[key]?'err':''}" placeholder="${ph}"
value="${esc(l[key])}">

    ${e[key]?`<div class="errmsg">⚠ ${e[key]}</div>`:''}

</div>`).join('')}

<div class="actions-row">

    <button class="btn btn-ghost" onclick="go('mode')">← Back</button>

    <div class="left-actions">

        <button class="btn-text" onclick="go('privacy')">Skip this step</button>

        <button class="btn btn-primary" onclick="submitLinks()">Continue →</button>

    </div>

</div>

</div>`;
}

function submitLinks(){
    const l = S.links;

    ['github','linkedin','portfolio','behance'].forEach(k=> l[k] = val('f-'+k));

    const e = {};

    ['github','linkedin','portfolio','behance'].forEach(k=>{ if(l[k] && !isValidUrl(l[k])) e[k] = "Please enter
a valid URL."; });

    S.errors = e;

    if(Object.keys(e).length){ render(); return; }

    go('privacy');
}

/* ===== PRIVACY ===== */

function renderPrivacy(){
    const p = S.privacy;

    const vis = ["Public","Campus Only","Private"];

    return `

    <div class="card">

```

```
<div class="eyebrow">Privacy</div>
```

```
<h1 class="step-title">Choose what others can see</h1>
```

```
<p class="step-sub">You're always in control of your visibility on SkillSwap Campus.</p>
```

```
<div class="field-group">
```

```
  <label class="field-label">Profile visibility</label>
```

```
  <div class="chip-field">
```

```
    ${vis.map(v=>`<div class="chip ${p.visibility===v?'active':''}" onclick="S.privacy.visibility='${v}';
render();">${v}</div>`).join("")}
```

```
  </div>
```

```
</div>
```

```
<div style="margin-top:24px">
```

```
  ${toggleRow('showEmail','Show email on profile','Off by default to protect your inbox')}
```

```
  ${toggleRow('showLinks','Show portfolio & social links','Only shown if you added links')}
```

```
  ${toggleRow('showAvailability','Show availability','Helps partners know when to reach you')}
```

```
</div>
```

```
<div class="actions-row">
```

```
  <button class="btn btn-ghost" onclick="go('links')">← Back</button>
```

```
  <button class="btn btn-primary" onclick="go('preview')">Continue →</button>
```

```
</div>
```

```
</div>;
```

```
}
```

```
function toggleRow(key, label, sub){
```

```
  const checked = S.privacy[key];
```

```
  return `<div class="toggle-row">
```

```
    <div><div class="tr-label">${label}</div><div class="tr-sub">${sub}</div></div>
```

```
    <label class="switch"><input type="checkbox" ${checked?'checked':''}>
```

```
    onchange="S.privacy['${key}']=this.checked; render();"><span class="slider"></span></label>
```

```
  </div>;
```

```
}
```

```

/* ===== BADGES & TRUST SCORE =====
*/

function renderBadges(){

  const badges = [];

  badges.push(S.emailVerified ? `<div class="pill" style="border-color:rgba(52,211,153,0.4);
color:#8ff0c8">✓ Email Verified</div>` : `<div class="pill" style="color:var(--muted-dim)">Email
Unverified</div>`);

  if(S.phoneVerified) badges.push(`<div class="pill" style="border-color:rgba(52,211,153,0.4);
color:#8ff0c8">✓ Phone Verified</div>`);

  if(S.collegeVerified===`verified`) badges.push(`<div class="pill" style="border-
color:rgba(52,211,153,0.4); color:#8ff0c8">✓ College Verified</div>`);

  else if(S.collegeVerified===`manual`) badges.push(`<div class="pill" style="color:var(--warn); border-
color:rgba(251,191,36,0.35)">⌚ College ID in review</div>`);

  if(S.emailVerified && S.phoneVerified && S.collegeVerified===`verified`) badges.push(`<div
class="pill" style="border-color:rgba(59,130,246,0.5); color:#bcdcff">🎓 Verified Student</div>`);

  return badges.join("");
}

function computeTrustScore(){

  // Mock scoring — in a real system this comes from swap history, ratings, response times, etc.

  const t = S.trust;

  let base = 60;

  if(S.emailVerified) base += 8;

  if(S.phoneVerified) base += 7;

  if(S.collegeVerified===`verified`) base += 10;

  base += Math.round(t.responseRate/10);

  base -= Math.round(t.cancellationRate/2);

  return Math.max(0, Math.min(99, base));
}

/* ===== COMPLETION SCORE ===== */

function computeCompletion(){

  let score = 0; const suggestions = [];

```

```

// Basic info 15%
if(S.account.fullName && S.profile.username) score += 15;

// Academic 15%
if(S.academic.college && S.academic.degree && S.academic.degree!="Select...") score += 15;

// Photo 10%
if(S.profile.photo) score += 10; else suggestions.push("Add a profile photo +10%");

// Bio 10%
if(S.about.bio && S.about.bio.length>=150) score += 10;

// Teaching skills 20%
if(Object.keys(S.teach).length>0) score += 20;

// Learning skills 20%
if(Object.keys(S.learn).length>0) score += 20;

// Availability 5%
if(S.availability.days.length>0) score += 5;

// Portfolio 5%
const hasLink = Object.values(S.links).some(v=>v && v.trim());
if(hasLink) score += 5; else suggestions.push("Add your GitHub +5%");

return {score, suggestions};
}

/* ===== PREVIEW ===== */
function renderPreview(){
  const initials = (S.account.fullName || "?").split(' ').map(w=>w[0]).join('').slice(0,2).toUpperCase();
  const {score, suggestions} = computeCompletion();
  return `
<div class="card">
  <div class="eyebrow">Profile preview</div>
  <h1 class="step-title">Here's your SkillSwap profile</h1>
  <p class="step-sub">This is exactly how other students will see you. You can edit any section
before going live.</p>

```

```

<div class="completion-box">

  <svg class="ring" viewBox="0 0 56 56"><circle cx="28" cy="28" r="24" fill="none"
stroke="rgba(255,255,255,0.08)" stroke-width="6"/>

  <circle cx="28" cy="28" r="24" fill="none" stroke="url(#ringgrad)" stroke-width="6" stroke-
linecap="round"

    stroke-dasharray="${2*Math.PI*24}" stroke-dashoffset="${2*Math.PI*24*(1-score/100)}"
transform="rotate(-90 28 28)"/>

  <defs><linearGradient id="ringgrad" x1="0" y1="0" x2="1" y2="1"><stop offset="0" stop-
color="#3b82f6"/><stop offset="1" stop-color="#22d3ee"/></linearGradient></defs>

</svg>
<div>

  <div style="font-weight:700; font-family:var(--font-display)">Your profile is ${score}%
complete</div>

  ${ suggestions.length ? `<div class="completion-suggest">${ suggestions.join(' &nbsp;·&nbsp;
')}</div>` : `<div class="completion-suggest">Nice — everything's filled in.</div>` }

</div>
</div>

<div class="preview-hero">

  <div class="preview-avatar">${ S.profile.photo ? `` : initials}</div>

  <div>

    <div class="preview-name">${ esc(S.account.fullName) } | 'Your Name'</div>

    <div class="preview-username">@${ esc(S.profile.username) } | 'username'</div>

    <div class="preview-meta">${ esc(S.academic.college) } | 'College' ·
    ${ esc(S.academic.degree) }&&S.academic.degree!=='Select...'?esc(S.academic.degree):'Degree'
    ${ S.academic.year&&S.academic.year!=='Select...'?· ' '+esc(S.academic.year):''}</div>

    <div class="pill-row" style="margin-top:9px">${ renderBadges()}</div>

  </div>
</div>

<div class="preview-section">

  <div class="preview-section-head"><h3>About</h3><button class="edit-link"
onclick="go('about')">Edit</button></div>

  <p class="bio-text">${ esc(S.about.bio) } | 'No bio added yet.'</p>

```

```
    ${S.about.languages.length ? `<div class="pill-row" style="margin-
top:10px">${S.about.languages.map(l=>`<div class="pill">${l}</div>`).join("")}</div>` : ""}

</div>
```

```
<div class="preview-section">

  <div class="preview-section-head"><h3>I can teach</h3><button class="edit-link"
onclick="go('teach')">Edit</button></div>

  <div class="pill-row">${Object.entries(S.teach).map(([n,l])=>`<div class="pill teach">${esc(n)} •
${l}</div>`).join("")} || '<span class="hint">Nothing added yet</span>'}</div>

</div>
```

```
<div class="preview-section">

  <div class="preview-section-head"><h3>I want to learn</h3><button class="edit-link"
onclick="go('learn')">Edit</button></div>

  <div class="pill-row">${Object.entries(S.learn).map(([n,d])=>`<div class="pill learn">${esc(n)} •
${d.level}</div>`).join("")} || '<span class="hint">Nothing added yet</span>'}</div>

</div>
```

```
<div class="preview-section">

  <div class="preview-section-head"><h3>Goals & matching preferences</h3><button
class="edit-link" onclick="go('purpose')">Edit</button></div>

  <div class="pill-row">${S.purpose.map(p=>`<div class="pill">${esc(p)}</div>`).join("")} || '<span
class="hint">Not set</span>'}</div>

  <p class="bio-text" style="margin-top:8px">Prefers: ${S.partnerPref.network},
${S.partnerPref.course}</p>

</div>
```

```
<div class="preview-section">

  <div class="preview-section-head"><h3>Availability & mode</h3><button class="edit-link"
onclick="go('availability')">Edit</button></div>

  <p class="bio-text">${S.availability.days.join(', ')} || 'Not set' · ${S.availability.useCustom ?
`${S.availability.customFrom}–${S.availability.customTo}` : S.availability.time} ·
${S.availability.duration} sessions<br>Mode: ${S.learningMode.join(', ')} || 'Not set' · Contact:
${S.communication.join(', ')} || 'Not set'</p>

</div>
```

```

    ${Object.values(S.links).some(v=>v)} ? `

    <div class="preview-section">

      <div class="preview-section-head"><h3>Links</h3><button class="edit-link"
onclick="go('links')">Edit</button></div>

      <div class="pill-row">${Object.entries(S.links).filter(([k,v])=>v).map(([k,v])=>`<div
class="pill">${k}</div>`).join("")}</div>

    </div>` : ""}

    <div class="actions-row" style="justify-content:flex-end">

      <button class="btn btn-primary btn-block" onclick="completeProfile()">Complete My Profile
🔗</button>

    </div>

</div>;

}

function completeProfile(){ go('success'); }

/* ===== SUCCESS ===== */

function renderSuccess(){

  return `

    <div class="stage" style="max-width:560px">

      <div class="card success-wrap">

        <svg class="success-check" viewBox="0 0 76 76">

          <circle cx="38" cy="38" r="34" fill="none" stroke="url(#sg)" stroke-width="4"/>

          <path d="M24 39l10 18-20" fill="none" stroke="url(#sg)" stroke-width="5" stroke-
linecap="round" stroke-linejoin="round" stroke-dasharray="60" stroke-dashoffset="60"
style="animation: draw 0.7s 0.3s ease forwards"/>

          <defs><linearGradient id="sg" x1="0" y1="0" x2="1" y2="1"><stop offset="0" stop-
color="#3b82f6"/><stop offset="1" stop-color="#22d3ee"/></linearGradient></defs>

          <style>@keyframes draw{to{stroke-dashoffset:0;}}</style>

        </svg>

        <h1 class="step-title" style="margin-bottom:6px">Your SkillSwap profile is ready! 🔗</h1>

```

<p class="step-sub" style="margin:0 auto 20px">You're now ready to teach, learn and swap skills with students around you.</p>

<div class="eyebrow" style="justify-content:center; margin-bottom:4px">Profile completion: 100%</div>

<div class="success-actions">

<div class="success-action" onclick="go('dashboard')"><div class="sa-icon"></div><div class="sa-title">Find Skill Partners</div></div>

<div class="success-action" onclick="go('dashboard')"><div class="sa-icon"></div><div class="sa-title">Explore Skills</div></div>

<div class="success-action" onclick="go('dashboard')"><div class="sa-icon"></div><div class="sa-title">Go to Dashboard</div></div>

</div>

<button class="btn btn-primary btn-block" onclick="go('dashboard')">Go to Dashboard
-></button>

</div>

</div>;

}

/* ===== DASHBOARD + MATCHING
===== */

const MOCK_STUDENTS = [

{name:"Rahul Verma", college:"IIT Patna", course:"B.Tech", teach:["JavaScript","React"],
learn:["UI/UX"], days:["Mon","Wed","Fri"], mode:["Online"], languages:["English","Hindi"],
purpose:["Find project partners","Build portfolio"]},

{name:"Sneha Kapoor", college:"NIT Trichy", course:"B.Tech", teach:["Photography","Photoshop"],
learn:["Python"], days:["Sat","Sun"], mode:["Offline"], languages:["English","Tamil"],
purpose:["Learn new skills","Meet other students"]},

{name:"Aman Yadav", college:"IIT Patna", course:"B.Tech", teach:["UI/UX","Graphic Design"],
learn:["JavaScript","Git/GitHub"], days:["Tue","Thu"], mode:["Both"], languages:["English","Hindi"],
purpose:["Build portfolio","Prepare for career"]},

{name:"Divya Rao", college:"BITS Pilani", course:"BBA", teach:["Public Speaking"], learn:["Video
Editing"], days:["Mon","Tue","Sat"], mode:["Online"], languages:["English"], purpose:["Meet other
students"]},

{name:"Karan Mehta", college:"IIT Patna", course:"B.Tech", teach:["Python","C++"],
learn:["Photography"], days:["Wed","Fri","Sun"], mode:["Both"], languages:["English","Hindi"],
purpose:["Teach my skills","Academic help"]},

```

    {name:"Fatima Sheikh", college:"DU", course:"BA", teach:["Digital Marketing"], learn:["Web Development"], days:["Thu","Fri"], mode:["Online"], languages:["English","Hindi"], purpose:["Prepare for career"]},
];

function computeMatch(student){

    const myTeach = Object.keys(S.teach);

    const myLearn = Object.keys(S.learn);

    let score = 20; // baseline


    // Core skill compatibility (heaviest weight — the whole point of a swap)

    const teachOverlap = student.teach.filter(s=>myLearn.includes(s)); // they teach what I want to learn

    const learnOverlap = student.learn.filter(s=>myTeach.includes(s)); // they want to learn what I teach

    score += teachOverlap.length * 18;

    score += learnOverlap.length * 18;


    // Availability compatibility

    const sharedDays = student.days.filter(d=>S.availability.days.includes(d));

    if(sharedDays.length) score += Math.min(sharedDays.length * 3, 9);


    // Learning mode compatibility

    const modeMatch = student.mode.some(m=> m==="Both" || S.learningMode.includes(m) || S.learningMode.includes("Both"));

    if(modeMatch) score += 6;


    // Language overlap

    const sharedLangs = student.languages.filter(l=>S.about.languages.includes(l));

    if(sharedLangs.length) score += 4;


    // College / course proximity, respecting the user's stated preference

    const sameCollege = student.college === S.academic.college;

    const sameCourse = student.course === S.academic.degree;

```

```

if(S.partnerPref.network==="Same college" ? sameCollege : true) score += sameCollege ? 6 : 0;
if(S.partnerPref.course==="Same course" ? sameCourse : true) score += sameCourse ? 4 : 0;

// Shared goals
const sharedGoals = student.purpose.filter(p=>S.purpose.includes(p));
if(sharedGoals.length) score += 3;

score = Math.max(10, Math.min(score, 99));
const perfect = teachOverlap.length>0 && learnOverlap.length>0;
return {score, perfect, teachOverlap, learnOverlap, sharedDays, sharedLangs};
}

function renderDashboard(){
  const name = (S.account.fullName || "there").split(' ')[0];
  const hour = new Date().getHours();
  const greet = hour<12 ? "Good morning" : hour<18 ? "Good afternoon" : "Good evening";
  const matches = MOCK_STUDENTS.map(s=>({...s, m: computeMatch(s)})).sort((a,b)=>b.m.score - a.m.score);

  const trustScore = computeTrustScore();
  return `
<div class="dash-header">
  <div class="dash-greeting">
    <h1>${greet}, ${esc(name)} 🐼</h1>
    <p>Ready to learn something new or share your skills?</p>
    <div class="pill-row" style="margin-top:10px">${renderBadges()}</div>
  </div>
  <div style="display:flex; gap:10px; align-items:center; flex-wrap:wrap">
    <div class="stat-card" style="padding:12px 18px; text-align:center">
      <div class="num" style="font-size:22px">${trustScore}</div>
      <div class="lbl">Trust Score</div>
    </div>

```

```

    <button class="btn btn-ghost" onclick="go('admin')">Admin Panel</button>

    <button class="btn btn-ghost" onclick="if(confirm('Restart the onboarding demo?'))
restartDemo()">Restart demo</button>

</div>

</div>

<div class="stat-grid">

    <div class="stat-card"><div class="num">${Object.keys(S.teach).length}</div><div
class="lbl">Your skills</div></div>

    <div class="stat-card"><div class="num">${Object.keys(S.learn).length}</div><div
class="lbl">Want to learn</div></div>

    <div class="stat-card"><div class="num">${matches.filter(m=>m.m.score>=70).length}</div><div
class="lbl">Skill matches</div></div>

    <div class="stat-card"><div class="num">3</div><div class="lbl">Pending requests</div></div>

</div>

<div class="section-head"><h2>Recommended skill partners</h2></div>

<div class="match-grid">

    ${matches.filter(s=>!S.blockedUsers.includes(s.name)).map(s=>{

        const initials = s.name.split(' ').map(w=>w[0]).join('').slice(0,2);

        const safeName = s.name.replace(/'/g, "\\'");

        return `<div class="match-card">

            <div class="match-top">

                <div class="match-avatar">${initials}</div>

                <div><div class="match-name">${s.name}</div><div class="match-
college">${s.college}</div></div>

                <div class="match-score"><b>${s.m.score}%</b><span>match</span></div>

            </div>

            <div class="match-row-label">Can teach</div>

            <div class="pill-row">${s.teach.map(t=>`<div class="pill teach">${t}</div>`).join("")}</div>

            <div class="match-row-label">Wants to learn</div>

            <div class="pill-row">${s.learn.map(t=>`<div class="pill learn">${t}</div>`).join("")}</div>

            ${s.m.perfect ? `<div class="perfect-tag">🔥 Perfect SkillSwap match</div>` : ""}

```

```

    <div class="match-actions">

      <button class="btn btn-ghost" onclick="toast('Viewing ${safeName}\\'s profile (demo)')">View
Profile</button>

      <button class="btn btn-primary" onclick="toast('Swap request sent to ${safeName}
(demo)')">Send Request</button>

    </div>

    <div style="display:flex; gap:14px; margin-top:10px; justify-content:center">

      <button class="btn-text" style="font-size:11px"
onclick="reportUser('${safeName}')">Report</button>

      <button class="btn-text" style="font-size:11px"
onclick="blockUser('${safeName}')">Block</button>

    </div>

  </div>`;
  }).join("")
</div>;
}

function reportUser(name){
  S.reportedUsers.push({name, at: new Date().toLocaleString()});
  toast(`${name} reported — our safety team will review this (demo).`);
}

function blockUser(name){
  S.blockedUsers.push(name);
  toast(`${name} blocked — you won't see them again.`);
  render();
}

function renderAdmin(){
  return `
  <div class="card">

    <div class="eyebrow">Admin panel · demo</div>

    <h1 class="step-title">Reported users</h1>

    <p class="step-sub">A real admin panel would let moderators review evidence and take action.
This is a simplified view for the prototype.</p>

    ${S.reportedUsers.length ? S.reportedUsers.map(r=>`

```

```

    <div class="selected-item"><div><div class="name">${esc(r.name)}</div><div
class="hint">Reported ${esc(r.at)}</div></div></div>

    `).join(") : `<div class="empty-note">No reports yet.</div>`

    <div class="selected-panel-title" style="margin-top:24px">Blocked by you<span
class="count">${S.blockedUsers.length}</span></div>

    ${S.blockedUsers.length ? S.blockedUsers.map(n=>`<div class="pill" style="margin-
right:6px">${esc(n)}</div>`).join(") : `<div class="empty-note">Nothing blocked.</div>`}

    <div class="actions-row" style="justify-content:flex-end; margin-top:24px">

        <button class="btn btn-primary" onclick="go('dashboard')">Back to Dashboard →</button>

    </div>
</div>`;
}

function restartDemo(){
    location.reload();
}

/* ===== AFTER-RENDER HOOKS ===== */
function afterRender(step){
    if(step==='verify'){
        const first = document.getElementById('otp-0');
        if(first) first.focus();
    }

    document.querySelectorAll('input[type=text], input[type=email], input[type=password],
textarea').forEach(el=>{
        el.addEventListener('keydown', e=>{
            if(e.key==='Enter' && el.tagName!=='TEXTAREA'){
                const btn = el.closest('.card')?.querySelector('.btn-primary');
                if(btn) btn.click();
            }
        });
    });
}

```

```
render();
```

```
</script>
```

```
</body>
```

```
</html>
```